Database Management Systems

A Comprehensive Reference

Compiled Reference Guide

2026

Contents

1 Introduction to Database Management Systems **4**
- 1.1 What is a Database? . . . 4
- 1.2 Characteristics of the Database Approach . . . 4
- 1.3 Advantages of Using a DBMS . . . 5
- 1.4 Users of a Database System . . . 5
- 1.5 Three-Schema Architecture . . . 5
- 1.6 Database Languages . . . 6
- 1.7 Types of Database Management Systems . . . 6
- 1.8 Database System Architectures . . . 6

2 Data Models and Database Design **7**
- 2.1 What is a Data Model? . . . 7
- 2.2 The Entity-Relationship (ER) Model . . . 7
- 2.3 Converting an ER Diagram to Relations . . . 8
- 2.4 Database Design Process . . . 8

3 The Relational Model **10**
- 3.1 Basic Concepts . . . 10
- 3.2 Relational Model Constraints . . . 10
- 3.3 Keys . . . 10
- 3.4 Integrity Constraints and Referential Actions . . . 10
- 3.5 Relational Algebra . . . 11
- 3.6 Relational Calculus . . . 11

4 SQL — Structured Query Language **13**
- 4.1 Overview . . . 13
- 4.2 Data Definition Language (DDL) . . . 13
- 4.3 Data Manipulation Language (DML) . . . 13
- 4.4 Views . . . 16
- 4.5 Indexes . . . 16
- 4.6 Data Control Language (DCL) . . . 16
- 4.7 Transaction Control Language (TCL) . . . 16
- 4.8 Stored Procedures, Functions, and Triggers . . . 16

4.9 SQL Query Execution Order (Logical Processing Order) 17

5 Functional Dependencies and Normalization 18
5.1 Functional Dependencies (FDs) 18
5.2 Anomalies from Poor Design 18
5.3 Normal Forms 18
5.4 Multivalued Dependencies and Join Dependencies 19
5.5 Decomposition Properties 19
5.6 Denormalization 20

6 Transaction Management 21
6.1 What is a Transaction? 21
6.2 ACID Properties 21
6.3 Transaction States 21
6.4 Schedules 21
6.5 Concurrency Control 22
6.6 Isolation Levels (SQL Standard) 22
6.7 Recovery Management 23

7 Storage, File Organization, and Indexing 25
7.1 Storage Hierarchy 25
7.2 File Organization 25
7.3 Indexing 25
7.4 Query Processing and Optimization 26

8 Distributed Databases and Parallel Databases 28
8.1 Distributed Database Concepts 28
8.2 Distributed Transaction Management 28
8.3 The CAP Theorem 29
8.4 Parallel Databases 29

9 NoSQL and Modern Database Systems 30
9.1 Why NoSQL? 30
9.2 Categories of NoSQL Databases 30
9.3 NewSQL 30
9.4 Data Warehousing and OLAP 30

10 Database Security 32
10.1 Security Threats 32
10.2 Authentication and Authorization 32
10.3 SQL Injection 32
10.4 Encryption 33
10.5 Auditing 33
10.6 Backup and Recovery Strategy 33

11 Popular Relational Database Systems — A Comparative Overview 34
11.1 MySQL 34
11.2 PostgreSQL 34
11.3 Oracle Database 34

11.4 Microsoft SQL Server 34
11.5 SQLite 34
11.6 Comparison Summary 34

12 Advanced and Emerging Topics **36**
12.1 Object-Relational and Object-Oriented Databases 36
12.2 Temporal Databases 36
12.3 Spatial and Geographic Databases 36
12.4 In-Memory Databases 36
12.5 Graph Databases in Depth 36
12.6 Data Mining and Big Data Integration 36
12.7 Database Trends 37

13 Solved Problems and Worked Examples **38**
13.1 Problem Set 1: ER Modeling 38
13.2 Problem Set 2: Normalization 39
13.3 Problem Set 3: Relational Algebra 40
13.4 Problem Set 4: Transaction Schedules and Serializability 40
13.5 Problem Set 5: Deadlock Detection 40
13.6 Problem Set 6: Two-Phase Commit Walkthrough 40
13.7 Problem Set 7: Index Selection 41

Appendix A: Glossary of Key Terms **42**

Appendix B: Quick Reference — Common SQL Commands **43**

1 Introduction to Database Management Systems

1.1 What is a Database?

A **database** is an organized collection of structured, related data that is stored and accessed electronically. A **Database Management System (DBMS)** is the software that enables users to define, create, maintain, and control access to the database. Together, the database and DBMS (along with associated applications) are referred to as a **database system**.

Before DBMSs became widespread, organizations relied on **file-based systems**, where each application maintained its own files. This approach suffered from several serious problems:

- **Data redundancy and inconsistency**: The same data stored in multiple files could easily become inconsistent when updated in one place but not another.
- **Difficulty accessing data**: New queries often required writing new application programs.
- **Data isolation**: Data scattered across files in different formats made it hard to write programs to retrieve data.
- **Integrity problems**: Constraints (e.g., "account balance must not go below zero") had to be coded into every application program separately.
- **Atomicity problems**: If a system failed mid-transaction, data could be left in an inconsistent state.
- **Concurrent access anomalies**: Multiple users accessing and updating data simultaneously could interfere with each other, leading to inconsistent results.
- **Security problems**: It was difficult to enforce access restrictions when every application had its own ad-hoc file access logic.

A DBMS solves these problems by providing a centralized, controlled repository for data with well-defined mechanisms for access, integrity, concurrency, and recovery.

1.2 Characteristics of the Database Approach

1. **Self-describing nature**: A database system contains not only the data itself but also a complete description of the database structure (metadata), stored in a system catalog.
2. **Insulation between programs and data (data abstraction)**: The structure of data is stored separately from the programs that access it, so changes to structure don't necessarily require changing every program.
3. **Support for multiple views of data**: Different users can have different, tailored views of the database.
4. **Data sharing and multi-user transaction processing**: Many users can access the database at the same time, with the DBMS ensuring correctness through concurrency control.
5. **Control of redundancy**: A well-designed database minimizes duplicate storage of the same information.
6. **Enforcement of integrity constraints**: Rules about what data is valid are enforced automatically by the DBMS.

7. **Restriction of unauthorized access**: Security and authorization subsystems ensure only permitted users can perform permitted operations.

1.3 Advantages of Using a DBMS

- Controlling redundancy
- Restricting unauthorized access
- Providing persistent storage for program objects
- Permitting efficient query processing
- Providing backup and recovery
- Providing multiple user interfaces
- Representing complex relationships among data
- Enforcing integrity constraints
- Permitting inference and actions using rules and triggers

1.4 Users of a Database System

- **Database Administrators (DBAs)**: Responsible for authorizing access, coordinating and monitoring use, acquiring resources, controlling schema and physical organization, and granting authorization.
- **Database Designers**: Identify the data to be stored and choose appropriate structures.
- **Application Programmers**: Write application programs that interact with the database, often using embedded SQL, APIs, or ORMs.
- **End Users**:
  - *Naive users*: Interact with the system through pre-written application programs (e.g., using an ATM interface).
  - *Sophisticated users*: Interact using query languages directly.
  - *Casual users*: Access the database occasionally with different requirements each time.

1.5 Three-Schema Architecture

The **ANSI/SPARC three-schema architecture** separates the user applications from the physical database through three levels:

1. **Internal (Physical) Level**: Describes the physical storage structure of the database — file organization, indexes, data compression, encryption.
2. **Conceptual (Logical) Level**: Describes the structure of the whole database for the community of users, hiding physical storage details. This is where entities, data types, relationships, and constraints are described.
3. **External (View) Level**: Includes user views, each describing the part of the database that a particular user group is interested in.

1.5.1 Data Independence

The three-schema architecture enables two types of **data independence**:

- **Logical data independence**: The ability to change the conceptual schema without changing external schemas or application programs.
- **Physical data independence**: The ability to change the internal schema without changing the conceptual schema (e.g., reorganizing files, adding indexes).

1.6 Database Languages

- **DDL (Data Definition Language)**: Used to define schema (`CREATE`, `ALTER`, `DROP`).
- **DML (Data Manipulation Language)**: Used to manipulate data (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- **DCL (Data Control Language)**: Used to control access (`GRANT`, `REVOKE`).
- **TCL (Transaction Control Language)**: Used to manage transactions (`COMMIT`, `ROLLBACK`, `SAVEPOINT`).

1.7 Types of Database Management Systems

Type	Description	Examples
Hierarchical	Tree-structured, parent-child records	IMS
Network	Graph-structured, records linked via pointers	IDMS
Relational	Data in tables (relations) with rows and columns	MySQL, PostgreSQL, Oracle, SQL Server
Object-Oriented	Data stored as objects, as in OOP	db4o, ObjectDB
Object-Relational	Relational with object-oriented extensions	PostgreSQL
NoSQL	Non-relational, schema-flexible, horizontally scalable	MongoDB, Cassandra, Redis
NewSQL	Combines relational guarantees with NoSQL scalability	Google Spanner, CockroachDB

1.8 Database System Architectures

- **Centralized**: All processing done on a single central system.
- **Client-Server**: The DBMS server handles storage and queries; clients handle presentation logic. Can be one-tier, two-tier, or three-tier.
- **Distributed**: Data is stored across multiple physical locations, appearing as a single database to users.
- **Parallel**: Uses multiple CPUs/disks to improve performance and provide high availability.

2 Data Models and Database Design

2.1 What is a Data Model?

A **data model** is a collection of concepts used to describe the structure of a database — the data types, relationships, and constraints. Data models can be classified into:

- **High-level (conceptual) data models**: Provide concepts close to how users perceive data (e.g., Entity-Relationship model).
- **Representational (implementation) data models**: Provide concepts understandable by end users but not too far from the way data is organized in storage (e.g., the relational model).
- **Low-level (physical) data models**: Describe how data is stored in the computer, including record formats, orderings, and access paths.

2.2 The Entity-Relationship (ER) Model

The **ER model** is the most widely used conceptual model for database design. Its main constructs are:

- **Entity**: A real-world object or concept that is distinguishable from other objects (e.g., a specific employee, a specific course).
- **Entity Set/Type**: A collection of similar entities (e.g., all employees).
- **Attribute**: A property that describes an entity (e.g., Name, Age, Salary).
  - *Simple vs. Composite*: Simple attributes cannot be divided further; composite attributes can (e.g., Address = Street + City + State).
  - *Single-valued vs. Multi-valued*: Multi-valued attributes can have several values (e.g., PhoneNumbers).
  - *Stored vs. Derived*: Derived attributes are computed from other attributes (e.g., Age from DateOfBirth).
- **Key Attribute**: An attribute (or set of attributes) whose values uniquely identify entities (e.g., EmployeeID).
- **Relationship**: An association among two or more entities (e.g., an Employee *WorksOn* a Project).
- **Relationship Degree**: Number of entity types participating — unary (recursive), binary, ternary.
- **Cardinality Ratio**: Describes the maximum number of relationship instances an entity can participate in — one-to-one (1:1), one-to-many (1:N), many-to-many (M:N).
- **Participation Constraint**: *Total* participation means every entity must participate in the relationship (shown with a double line); *partial* participation means participation is optional.
- **Weak Entity**: An entity type without its own key attribute; it depends on a related "owner" entity via an *identifying relationship* and uses a *partial key* combined with the owner's key.

2.2.1 Extended ER (EER) Concepts

- **Specialization**: The process of defining subclasses of an entity type based on distinguishing characteristics (top-down).
- **Generalization**: The reverse process — combining entity types that share common attributes into a superclass (bottom-up).
- **Disjoint vs. Overlapping constraint**: Whether an entity can belong to more than one subclass simultaneously.
- **Total vs. Partial specialization**: Whether every entity in the superclass must belong to some subclass.
- **Union/Category**: A subclass that represents a collection of objects from distinct superclasses (a subset of the union).

2.3 Converting an ER Diagram to Relations

1. **Strong (regular) entity**: Becomes a table; simple attributes become columns; the key attribute becomes the primary key.
2. **Weak entity**: Becomes a table whose primary key is the combination of its partial key and the owner entity's primary key (as a foreign key).
3. **Composite attribute**: Only the simple component attributes are represented as columns.
4. **Multi-valued attribute**: Becomes a separate table with a foreign key referencing the owner entity.
5. **1:1 relationship**: Add the primary key of one entity as a foreign key in the other (typically the one with total participation), or merge the two tables.
6. **1:N relationship**: The primary key of the "1" side becomes a foreign key in the table on the "N" side.
7. **M:N relationship**: Create a new table containing the primary keys of both participating entities as a composite foreign key (and any relationship attributes).
8. **Multi-way (ternary+) relationship**: Create a new table with foreign keys to all participating entities.
9. **Specialization/generalization**: Can be mapped using several strategies — one table per subclass with FK to superclass table, single table with a type discriminator column, or one table per subclass only (no superclass table), depending on constraints.

2.4 Database Design Process

1. **Requirements Analysis**: Gather and analyze what data needs to be stored and how it will be used.
2. **Conceptual Design**: Build an ER/EER model, independent of any specific DBMS.
3. **Logical Design**: Map the conceptual model to a logical data model (e.g., relational schema), and normalize it.
4. **Physical Design**: Decide on storage structures, indexes, partitioning, and access paths for performance.
5. **Implementation**: Create the schema using DDL and populate the database.

6. **Testing and Tuning**: Test with real workloads and refine indexes and structures.

3 The Relational Model

3.1 Basic Concepts

The **relational model**, introduced by E. F. Codd in 1970, represents data as a collection of **relations** (tables). Key terminology:

- **Relation**: A table with rows and columns.
- **Tuple**: A row in a relation, representing a single record.
- **Attribute**: A column in a relation, representing a property.
- **Domain**: The set of allowable values for an attribute.
- **Degree**: The number of attributes in a relation.
- **Cardinality**: The number of tuples in a relation.
- **Relation Schema**: The name of the relation plus its attributes, e.g., `Student(ID, Name, Major)`.
- **Relation Instance**: The actual data (set of tuples) in a relation at a given point in time.

3.2 Relational Model Constraints

- **Domain constraint**: Every value of an attribute must be from its defined domain.
- **Key constraint**: A relation cannot have two tuples with the same value in a candidate key.
- **Entity integrity constraint**: No attribute of the primary key can be null.
- **Referential integrity constraint**: A foreign key value must either match a primary key value in the referenced relation, or be null.
- **General semantic (business) constraints**: Domain-specific rules, e.g., "an employee's salary cannot exceed their manager's salary."

3.3 Keys

- **Superkey**: A set of attributes that uniquely identifies a tuple.
- **Candidate Key**: A minimal superkey (no subset of it is also a superkey).
- **Primary Key**: The candidate key chosen by the database designer to uniquely identify tuples; cannot contain NULL.
- **Alternate Key**: Candidate keys not chosen as the primary key.
- **Composite Key**: A primary key made up of two or more attributes.
- **Foreign Key**: An attribute (or set of attributes) in one relation that references the primary key of another (or the same) relation, enforcing referential integrity.
- **Surrogate Key**: An artificially generated key (e.g., an auto-incrementing integer) with no business meaning, used purely for identification.

3.4 Integrity Constraints and Referential Actions

When a foreign key relationship exists, the DBMS must decide what happens on `DELETE` or `UPDATE` of the referenced row:

- **CASCADE**: Automatically delete/update matching rows in the referencing table.

- **SET NULL**: Set the foreign key column(s) to NULL.
- **SET DEFAULT**: Set the foreign key column(s) to a default value.
- **RESTRICT / NO ACTION**: Reject the operation if matching rows exist.

3.5 Relational Algebra

Relational algebra is a procedural query language consisting of a set of operations that take one or two relations as input and produce a new relation as output.

3.5.1 Unary Operations

- **Selection (σ)**: Selects tuples that satisfy a given predicate. `σ(condition)(R)`
- **Projection (π)**: Selects specific columns, removing duplicates. `π(attributes)(R)`
- **Rename (ρ)**: Renames a relation and/or its attributes.

3.5.2 Set Operations (require union-compatible relations)

- **Union (∪)**: Tuples in R or S or both.
- **Intersection (∩)**: Tuples in both R and S.
- **Set Difference (−)**: Tuples in R but not in S.

3.5.3 Binary Operations

- **Cartesian Product (×)**: Combines every tuple of R with every tuple of S.
- **Join (⋈)**: Combines related tuples from two relations based on a condition.
  - *Theta Join*: Join with a general condition (θ).
  - *Equijoin*: Theta join where the condition uses only equality.
  - *Natural Join*: Equijoin on all common attributes, with duplicate columns removed.
  - *Outer Join*: Preserves unmatched tuples from one or both relations, padding with NULLs.
    - *Left Outer Join*: All tuples from the left relation.
    - *Right Outer Join*: All tuples from the right relation.
    - *Full Outer Join*: All tuples from both relations.
  - *Semi-join*: Returns tuples of R that have a match in S (only R's attributes).
- **Division (÷)**: Used for queries involving the phrase "for all," e.g., "find students who have taken all courses."

3.6 Relational Calculus

Relational calculus is a **declarative** (non-procedural) query language — it describes *what* to retrieve, not *how*.

- **Tuple Relational Calculus (TRC)**: Queries are expressed as `{t | P(t)}`, where `t` is a tuple variable and `P` is a formula.
- **Domain Relational Calculus (DRC)**: Queries are expressed using domain variables that range over attribute values instead of whole tuples.

Relational calculus is equivalent in expressive power to the “safe” subset of relational algebra — this equivalence is the basis of **Codd’s theorem**, which shows SQL-like declarative languages can express the same queries as procedural relational algebra.

4 SQL — Structured Query Language

4.1 Overview

SQL is the standard language for relational databases, combining DDL, DML, DCL, and TCL capabilities. It is declarative — you specify *what* data you want, not *how* to retrieve it.

4.2 Data Definition Language (DDL)

```sql
CREATE TABLE Employee (
    EmpID       INT PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    DeptID      INT,
    Salary      DECIMAL(10,2) CHECK (Salary > 0),
    HireDate    DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
        ON DELETE SET NULL
);

ALTER TABLE Employee ADD COLUMN Email VARCHAR(255) UNIQUE;
ALTER TABLE Employee DROP COLUMN Email;
ALTER TABLE Employee MODIFY COLUMN Salary DECIMAL(12,2);

DROP TABLE Employee;
TRUNCATE TABLE Employee;  -- removes all rows, resets identity, cannot be rolled back i
```

Common column constraints: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`, `AUTO_INCREMENT` / `IDENTITY`.

4.3 Data Manipulation Language (DML)

```sql
-- INSERT
INSERT INTO Employee (EmpID, Name, DeptID, Salary)
VALUES (101, 'Alice Chanda', 3, 8500.00);

-- SELECT
SELECT Name, Salary
FROM Employee
WHERE DeptID = 3 AND Salary > 5000
ORDER BY Salary DESC
LIMIT 10;

-- UPDATE
UPDATE Employee SET Salary = Salary * 1.10 WHERE DeptID = 3;

-- DELETE
```

```sql
DELETE FROM Employee WHERE EmpID = 101;
```

4.3.1 Filtering and Pattern Matching

```sql
SELECT * FROM Employee WHERE Name LIKE 'A%';       -- starts with A
SELECT * FROM Employee WHERE Salary BETWEEN 5000 AND 10000;
SELECT * FROM Employee WHERE DeptID IN (1, 2, 3);
SELECT * FROM Employee WHERE Email IS NULL;
```

4.3.2 Aggregate Functions and Grouping

```sql
SELECT DeptID, COUNT(*) AS NumEmployees, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY DeptID
HAVING COUNT(*) > 5
ORDER BY AvgSalary DESC;
```

Common aggregates: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. **`WHERE`** filters rows before grouping; **`HAVING`** filters groups after aggregation.

4.3.3 Joins in SQL

```sql
-- Inner join
SELECT e.Name, d.DeptName
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID;

-- Left outer join
SELECT e.Name, d.DeptName
FROM Employee e
LEFT JOIN Department d ON e.DeptID = d.DeptID;

-- Self join
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employee e1
JOIN Employee e2 ON e1.ManagerID = e2.EmpID;
```

4.3.4 Subqueries

```sql
-- Scalar subquery
SELECT Name FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);

-- Correlated subquery
SELECT Name FROM Employee e
```

```sql
WHERE Salary > (
    SELECT AVG(Salary) FROM Employee WHERE DeptID = e.DeptID
);

-- EXISTS
SELECT Name FROM Department d
WHERE EXISTS (SELECT 1 FROM Employee e WHERE e.DeptID = d.DeptID);
```

4.3.5 Set Operations

```sql
SELECT Name FROM Employee
UNION
SELECT Name FROM Contractor;          -- removes duplicates

SELECT Name FROM Employee
UNION ALL
SELECT Name FROM Contractor;          -- keeps duplicates

SELECT Name FROM Employee
INTERSECT
SELECT Name FROM Manager;

SELECT Name FROM Employee
EXCEPT
SELECT Name FROM Manager;             -- MINUS in Oracle
```

4.3.6 Window Functions

```sql
SELECT Name, DeptID, Salary,
       RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS SalaryRank,
       AVG(Salary) OVER (PARTITION BY DeptID) AS DeptAvgSalary
FROM Employee;
```

Common window functions: `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `NTILE()`, `LAG()`, `LEAD()`, `SUM()/AVG() OVER(...)`.

4.3.7 Common Table Expressions (CTEs)

```sql
WITH DeptAvg AS (
    SELECT DeptID, AVG(Salary) AS AvgSal
    FROM Employee GROUP BY DeptID
)
SELECT e.Name, e.Salary, d.AvgSal
FROM Employee e JOIN DeptAvg d ON e.DeptID = d.DeptID
WHERE e.Salary > d.AvgSal;
```

```sql
-- Recursive CTE (e.g., organizational hierarchy)
WITH RECURSIVE OrgChart AS (
    SELECT EmpID, Name, ManagerID, 1 AS Level
    FROM Employee WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmpID, e.Name, e.ManagerID, o.Level + 1
    FROM Employee e JOIN OrgChart o ON e.ManagerID = o.EmpID
)
SELECT * FROM OrgChart;
```

4.4 Views

```sql
CREATE VIEW HighEarners AS
SELECT Name, DeptID, Salary FROM Employee WHERE Salary > 10000;
```

Views provide logical data independence, simplify complex queries, and can restrict access to specific columns/rows. Some views are **updatable** (simple, single-table views); others are **read-only** (e.g., views with aggregates or joins).

4.5 Indexes

```sql
CREATE INDEX idx_emp_dept ON Employee(DeptID);
CREATE UNIQUE INDEX idx_emp_email ON Employee(Email);
DROP INDEX idx_emp_dept;
```

4.6 Data Control Language (DCL)

```sql
GRANT SELECT, INSERT ON Employee TO 'analyst'@'%';
REVOKE INSERT ON Employee FROM 'analyst'@'%';
```

4.7 Transaction Control Language (TCL)

```sql
START TRANSACTION;
UPDATE Account SET Balance = Balance - 500 WHERE AccID = 1;
UPDATE Account SET Balance = Balance + 500 WHERE AccID = 2;
COMMIT;
-- or ROLLBACK; to undo
SAVEPOINT sp1;
ROLLBACK TO sp1;
```

4.8 Stored Procedures, Functions, and Triggers

```sql
DELIMITER //
CREATE PROCEDURE GiveRaise(IN empId INT, IN pct DECIMAL(5,2))
```

```
BEGIN
    UPDATE Employee SET Salary = Salary * (1 + pct/100) WHERE EmpID = empId;
END //
DELIMITER ;

CREATE TRIGGER trg_salary_check
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < OLD.Salary THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot decrease';
    END IF;
END;
```

4.9 SQL Query Execution Order (Logical Processing Order)

Although you *write* SQL in the order `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY`, the engine *logically processes* it as:

1. `FROM` (and `JOIN`s)
2. `WHERE`
3. `GROUP BY`
4. `HAVING`
5. `SELECT`
6. `DISTINCT`
7. `ORDER BY`
8. `LIMIT / OFFSET`

This is why you cannot reference a `SELECT` alias in the `WHERE` clause but can in `ORDER BY`.

5 Functional Dependencies and Normalization

5.1 Functional Dependencies (FDs)

A **functional dependency** $X \to Y$ means that the value of attribute set X uniquely determines the value of attribute set Y. If two tuples agree on X, they must agree on Y.

5.1.1 Armstrong's Axioms (Inference Rules)

- **Reflexivity**: If $Y \subseteq X$, then $X \to Y$.
- **Augmentation**: If $X \to Y$, then $XZ \to YZ$.
- **Transitivity**: If $X \to Y$ and $Y \to Z$, then $X \to Z$.

Derived rules: **Union** ($X \to Y$, $X \to Z \implies X \to YZ$), **Decomposition** ($X \to YZ \implies X \to Y$ and $X \to Z$), **Pseudotransitivity** ($X \to Y$, $WY \to Z \implies WX \to Z$).

5.1.2 Closure of an Attribute Set (X^+)

The closure X^+ is the set of all attributes functionally determined by X, given a set of FDs. It's computed by repeatedly applying FDs whose left side is already known to be in the closure until no more attributes can be added. X^+ is used to test whether X is a candidate key (X^+ = all attributes) and whether a given FD is implied by a set of FDs.

5.1.3 Minimal (Canonical) Cover

A **minimal cover** of a set of FDs is an equivalent set where: every right-hand side is a single attribute, no FD can be removed without changing the closure, and no attribute can be removed from any left-hand side without changing the closure.

5.2 Anomalies from Poor Design

Without normalization, tables suffer from:

- **Insertion anomaly**: Cannot insert a fact about one entity without also having data about another (e.g., can't add a new course unless a student is enrolled in it, if course and student data are merged into one table).
- **Deletion anomaly**: Deleting a record inadvertently removes other important facts (e.g., deleting the last student in a course removes all knowledge of that course).
- **Update anomaly**: The same fact stored redundantly must be updated in every occurrence, risking inconsistency.

5.3 Normal Forms

Normal Form	Requirement
1NF	All attribute values are atomic (no repeating groups or multi-valued attributes); each column holds a single value.
2NF	In 1NF, and every non-prime attribute is fully functionally dependent on the *entire* primary key (no partial dependency on part of a composite key).
3NF	In 2NF, and no non-prime attribute is transitively dependent on the primary key (no non-prime attribute depends on another non-prime attribute).
BCNF (Boyce-Codd)	For every non-trivial FD X → Y, X must be a superkey. Stricter than 3NF.
4NF	In BCNF, and no non-trivial multivalued dependencies exist other than a candidate key determining all attributes.
5NF (PJNF)	Every join dependency is implied by the candidate keys; eliminates redundancy from join dependencies.

5.3.1 Worked Example

Consider `StudentCourse(StudentID, CourseID, StudentName, CourseName, Instructor)` with FDs: `StudentID → StudentName`, `CourseID → CourseName, Instructor`, and key `{StudentID, CourseID}`.

This violates 2NF because `StudentName` depends only on part of the key (`StudentID`), not the whole key. Decomposing into `Student(StudentID, StudentName)` and `Course(CourseID, CourseName, Instructor)` and a linking table `Enrollment(StudentID, CourseID)` resolves the partial dependency and removes redundancy.

5.4 Multivalued Dependencies and Join Dependencies

- **Multivalued Dependency (MVD)**: X →→ Y holds if, for a given X, the set of associated Y values is independent of the other attributes. Arises when a table incorrectly combines two independent 1:N relationships (e.g., an Employee's Skills and Languages in one table causes a combinatorial explosion of rows).
- **Join Dependency (JD)**: A generalization stating that a relation can be losslessly reconstructed by joining several of its projections. 5NF requires that all join dependencies follow from candidate keys.

5.5 Decomposition Properties

A good decomposition of a relation must satisfy:

1. **Lossless-join decomposition**: Joining the decomposed relations must reconstruct exactly the original relation (no spurious tuples). Guaranteed if the common attribute set of the two decomposed relations is a superkey of at least one of them.
2. **Dependency preservation**: All functional dependencies from the original relation should be enforceable without needing to join relations back together.

Note: it is not always possible to achieve both lossless-join and dependency-preserving decomposition simultaneously when converting to BCNF — 3NF decomposition can always achieve both, but BCNF sometimes sacrifices dependency preservation.

5.6 Denormalization

Denormalization is the deliberate introduction of redundancy into a normalized schema to improve read performance (fewer joins), often used in data warehouses and reporting systems, at the cost of storage space and increased risk of update anomalies. This is a design trade-off, not a mistake — OLAP systems often favor denormalized star schemas for query speed.

6 Transaction Management

6.1 What is a Transaction?

A **transaction** is a logical unit of work that accesses and possibly modifies database contents, comprising one or more operations, and must be executed either completely or not at all.

6.2 ACID Properties

- **Atomicity**: A transaction is an all-or-nothing unit — either all its operations complete, or none do. Managed via logs and rollback.
- **Consistency**: A transaction takes the database from one consistent state to another, preserving all integrity constraints.
- **Isolation**: Concurrently executing transactions must not interfere with each other; the result must be as if transactions executed serially.
- **Durability**: Once a transaction commits, its changes persist even in the event of a system failure.

6.3 Transaction States

A transaction moves through the following states: **Active** → (**Partially Committed** → **Committed**) or (**Failed** → **Aborted**). From Aborted, a transaction may either be restarted or killed.

6.4 Schedules

A **schedule** is a sequence of interleaved operations from multiple transactions.

- **Serial Schedule**: Transactions execute one after another with no interleaving — always consistent, but no concurrency benefit.
- **Serializable Schedule**: A (possibly interleaved) schedule whose effect is equivalent to some serial schedule.
  - **Conflict serializability**: Determined by checking whether conflicting operations (two operations on the same data item, where at least one is a write) can be swapped to reach a serial schedule. Tested using a **precedence graph**: if the graph is acyclic, the schedule is conflict-serializable.
  - **View serializability**: A more general (and harder to test) notion — the schedule produces the same final result as some serial schedule regarding what each read sees.
- **Recoverable Schedule**: A transaction commits only after all transactions whose changes it read have committed.
- **Cascadeless Schedule**: Transactions only read values written by *committed* transactions, avoiding cascading rollbacks.
- **Strict Schedule**: A transaction can neither read nor write an item until the transaction that last wrote it has committed or aborted — this simplifies recovery.

6.5 Concurrency Control

Concurrency control mechanisms ensure isolation while allowing multiple transactions to run simultaneously.

6.5.1 Lock-Based Protocols

- **Binary Locks**: Simple locked/unlocked state.
- **Shared/Exclusive (S/X) Locks**: Shared locks allow concurrent reads; exclusive locks are needed for writes and block all other access.
- **Two-Phase Locking (2PL)**: Each transaction has a *growing phase* (acquiring locks) and a *shrinking phase* (releasing locks), with no new locks acquired once any lock is released. Guarantees conflict serializability.
  - **Strict 2PL**: All exclusive locks held until commit/abort — prevents cascading aborts.
  - **Rigorous 2PL**: All locks (shared and exclusive) held until commit/abort.
- **Deadlock**: Two or more transactions waiting indefinitely for locks held by each other.
  - *Prevention*: Wait-Die and Wound-Wait schemes based on transaction timestamps.
  - *Detection*: Build a wait-for graph; a cycle indicates deadlock. Resolve by aborting a victim transaction.
  - *Avoidance*: Acquire all locks needed upfront, or order resource acquisition consistently.

6.5.2 Timestamp-Based Protocols

Each transaction is assigned a unique timestamp at start. Conflicting operations are ordered by timestamp to ensure serializability equivalent to the timestamp order — no locking is needed, but this can lead to more aborts (starvation is prevented since timestamps only increase).

6.5.3 Optimistic Concurrency Control (Validation-Based)

Transactions execute without restriction, then go through a **validation phase** before committing, checking for conflicts with concurrently executing transactions. If validation fails, the transaction is rolled back. Effective when conflicts are rare.

6.5.4 Multiversion Concurrency Control (MVCC)

Instead of locking, the DBMS keeps multiple versions of a data item, allowing readers to access an older, consistent snapshot while writers create new versions. Widely used in modern DBMSs (e.g., PostgreSQL, MySQL InnoDB, Oracle) because readers never block writers and vice versa.

6.6 Isolation Levels (SQL Standard)

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible*
Serializable	Prevented	Prevented	Prevented

*Note: some implementations (e.g., InnoDB's Repeatable Read via MVCC) prevent phantom reads too, going beyond the strict standard definition.

- **Dirty Read**: Reading uncommitted data from another transaction.
- **Non-repeatable Read**: Re-reading a row returns different data because another transaction modified and committed it in between.
- **Phantom Read**: Re-executing a range query returns a different set of rows because another transaction inserted/deleted rows in between.

6.7 Recovery Management

The recovery subsystem ensures atomicity and durability despite system, transaction, or media failures.

6.7.1 Log-Based Recovery

The DBMS maintains a **log** recording every update: `<T, X, old-value, new-value>`.

- **Deferred Update (NO-UNDO/REDO)**: Updates are only written to the database after a transaction commits; on recovery, only REDO is needed for committed transactions.
- **Immediate Update (UNDO/REDO)**: Updates can be written to disk before commit, so both UNDO (for uncommitted transactions) and REDO (for committed ones) may be needed on recovery.
- **Write-Ahead Logging (WAL)**: The log record for an update must be written to stable storage *before* the corresponding data page is written to disk — essential for correct recovery.

6.7.2 Checkpoints

A **checkpoint** periodically flushes all modified buffers to disk and records checkpoint information in the log, reducing the amount of log that must be processed during recovery.

6.7.3 ARIES Recovery Algorithm

A widely used industrial-strength recovery algorithm with three phases:

1. **Analysis**: Scan the log from the last checkpoint to determine which transactions were active and which pages were dirty at the time of crash.
2. **Redo**: Reapply all logged updates (even for uncommitted transactions) to restore the database to its exact state at the time of the crash.

3. **Undo**: Roll back all transactions that were still active (not committed) at the time of the crash, in reverse order.

6.7.4 Shadow Paging

An alternative to logging: maintains two page tables (current and shadow); updates are made to new copies of pages, and the current page table is atomically switched to point to them only on commit — no explicit UNDO/REDO logging needed, but it fragments storage and complicates concurrent transactions.

7 Storage, File Organization, and Indexing

7.1 Storage Hierarchy

From fastest/smallest/most expensive to slowest/largest/cheapest: **CPU registers → cache → main memory (RAM) → SSD → magnetic disk → optical/tape (offline)**. Databases are designed around the fact that disk I/O is orders of magnitude slower than memory access, so minimizing disk accesses is central to performance.

7.2 File Organization

Data is stored in files composed of **records**, grouped into **blocks/pages** (the unit of disk I/O, e.g., 4KB or 8KB).

- **Heap (unordered) file**: Records placed wherever there is space. Fast insertion; slow search (O(n)).
- **Sequential (sorted) file**: Records physically sorted by a key. Fast range queries and binary search; slow insertion (may require reorganization).
- **Hash file**: Records placed based on a hash function applied to a key. Very fast equality lookups; poor for range queries.
- **Clustered file**: Records of a table (or related tables) are physically stored together based on a clustering attribute, speeding up range/group queries.

7.3 Indexing

An **index** is an auxiliary structure that speeds up data retrieval at the cost of extra storage and slower writes (since indexes must also be updated).

- **Primary Index**: Built on the ordering key field of a sequentially ordered file; one index entry per block (sparse).
- **Clustering Index**: Built on a non-key ordering field; one entry per distinct value.
- **Secondary Index**: Built on a non-ordering field; must be dense (one entry per record) since the data isn't physically sorted by that field.
- **Dense vs. Sparse Index**: Dense indexes have an entry for every search key value; sparse indexes have entries only for some (e.g., one per block), relying on the data being sorted.
- **Multilevel Index**: An index on an index, reducing the number of I/Os needed for a lookup on very large indexes.

7.3.1 B-Tree and B+-Tree Indexes

The **B+-tree** is the most widely used indexing structure in relational DBMSs:

- All actual data pointers are stored at the **leaf level**, which is organized as a linked list to allow fast sequential/range scans.
- Internal nodes contain only routing keys to guide the search — no data.
- The tree is always **balanced**: every leaf is at the same depth, guaranteeing O(log n) search, insert, and delete.

- Nodes are typically sized to match the disk block size, and have a high **fan-out** (many children per node), which keeps the tree shallow even for millions of records — usually 3-4 levels suffice for very large tables.

A plain **B-tree** (without the "+") stores data pointers in internal nodes as well as leaves, which can make range queries less efficient since scanning requires traversing back up and down the tree.

7.3.2 Hash Indexes

- **Static Hashing**: Fixed number of buckets; fast for equality lookup but performs poorly if the file grows (overflow chains) or shrinks.
- **Dynamic/Extendible Hashing**: Uses a directory that can grow, doubling as needed, avoiding long overflow chains.
- **Linear Hashing**: Grows the hash file gradually, one bucket at a time, without needing a directory.

7.3.3 Bitmap Indexes

Used for columns with low cardinality (few distinct values, e.g., Gender, Status). A bitmap (bit array) is maintained for each distinct value, with a bit per row indicating whether that row has the value. Extremely efficient for `AND`/`OR` combinations of conditions using bitwise operations — common in data warehouses.

7.3.4 Other Index Types

- **Composite (Multi-column) Index**: Built on more than one column; useful for queries filtering/sorting on a combination of columns, following the *leftmost prefix rule*.
- **Covering Index**: Contains all the columns needed to answer a query, so the engine never has to access the base table.
- **Full-text Index**: Optimized for searching within text content.
- **Spatial Index (R-tree, Quad-tree)**: Optimized for geometric/geographic data.

7.4 Query Processing and Optimization

7.4.1 Steps in Query Processing

1. **Parsing and translation**: The SQL query is parsed and translated into an internal representation (relational algebra expression).
2. **Optimization**: The query optimizer generates multiple equivalent execution plans and estimates their cost, choosing the cheapest.
3. **Evaluation**: The chosen execution plan (a tree of physical operators) is executed by the query execution engine.

7.4.2 Cost Estimation

The optimizer estimates cost primarily in terms of **disk I/O**, using statistics maintained in the system catalog: table sizes, number of distinct values, histograms of

value distributions, and index availability.

7.4.3 Join Algorithms

- **Nested-Loop Join**: For each tuple in the outer relation, scan the entire inner relation. Simple but potentially expensive — O(n × m) in the worst case; a *block nested-loop* variant reduces I/O by processing blocks at a time.
- **Index Nested-Loop Join**: Uses an index on the inner relation's join attribute to avoid a full scan for each outer tuple.
- **Sort-Merge Join**: Both relations are sorted on the join attribute, then merged in a single pass — efficient when data is already sorted or sorting cost is acceptable.
- **Hash Join**: Builds an in-memory hash table on the smaller relation's join attribute, then probes it while scanning the larger relation — very efficient for equijoins when memory permits.

7.4.4 Query Optimization Techniques

- **Heuristic (rule-based) optimization**: Applies transformation rules such as pushing selections and projections as early as possible (reducing intermediate result size), performing the most restrictive selection/join first, and combining Cartesian products with subsequent selections into joins.
- **Cost-based optimization**: Enumerates alternative plans (join orders, access paths) and picks the one with lowest estimated cost, often using dynamic programming to avoid recomputation for common sub-plans.
- **Materialized views**: Precomputed and stored query results that can be used to answer future queries faster, at the cost of needing to be refreshed as underlying data changes.

8 Distributed Databases and Parallel Databases

8.1 Distributed Database Concepts

A **distributed database** is a collection of logically related databases distributed across multiple physical sites connected by a network, appearing to users as a single database (**distribution transparency**).

8.1.1 Types of Transparency

- **Fragmentation transparency**: Users need not know that data is fragmented.
- **Replication transparency**: Users need not know that copies of data exist at multiple sites.
- **Location transparency**: Users need not know the physical location of data.

8.1.2 Data Fragmentation

- **Horizontal Fragmentation**: Splitting a table's rows across sites (e.g., customers by region).
- **Vertical Fragmentation**: Splitting a table's columns across sites, with a common key to allow reconstruction.
- **Mixed (Hybrid) Fragmentation**: A combination of both.

8.1.3 Data Replication

Storing copies of data at multiple sites improves availability and read performance but complicates consistency maintenance during updates.

- **Synchronous replication**: All copies updated as part of the same transaction — strong consistency, higher latency.
- **Asynchronous replication**: Copies updated after the transaction commits — better performance, risk of temporary inconsistency (eventual consistency).

8.2 Distributed Transaction Management

8.2.1 Two-Phase Commit (2PC) Protocol

Ensures atomicity of transactions across multiple sites:

1. **Phase 1 (Voting/Prepare)**: The coordinator asks all participating sites if they are ready to commit; each site responds "yes" (and writes a "ready" log record) or "no."
2. **Phase 2 (Decision)**: If all sites vote "yes," the coordinator sends a "commit" message; if any site votes "no" (or times out), it sends "abort." Sites then act accordingly and acknowledge.

2PC guarantees atomicity but is a **blocking protocol** — if the coordinator fails after Phase 1, participants may be stuck waiting.

8.2.2 Three-Phase Commit (3PC)

Adds an extra "pre-commit" phase to make the protocol non-blocking under certain failure assumptions, at the cost of additional message overhead.

8.3 The CAP Theorem

In a distributed system, it is impossible to simultaneously guarantee all three of:

- **Consistency**: Every read receives the most recent write or an error.
- **Availability**: Every request receives a (non-error) response, without guarantee it contains the most recent write.
- **Partition Tolerance**: The system continues to operate despite network partitions.

Since network partitions are unavoidable in real distributed systems, designers must choose to prioritize either **Consistency (CP systems)** or **Availability (AP systems)** when a partition occurs. Many NoSQL systems favor AP with **eventual consistency**; traditional distributed relational systems often favor CP.

8.4 Parallel Databases

Parallel databases use multiple processors/disks to improve performance and availability.

- **Shared-Memory Architecture**: Multiple CPUs share a single main memory and disk — simple but limited scalability.
- **Shared-Disk Architecture**: Multiple CPUs, each with private memory, share a common disk — good availability but memory-bus contention limits scale.
- **Shared-Nothing Architecture**: Each node has its own CPU, memory, and disk, communicating over a network — highest scalability, used by most modern distributed and big-data systems (e.g., Google Spanner, Cassandra).

8.4.1 Types of Parallelism

- **Interquery parallelism**: Different queries execute in parallel on different nodes.
- **Intraquery parallelism**: A single query is broken into sub-tasks that execute in parallel (e.g., partitioning a large join/scan across nodes).

9 NoSQL and Modern Database Systems

9.1 Why NoSQL?

NoSQL ("Not Only SQL") databases emerged to address limitations of relational databases for certain modern workloads: massive horizontal scale, flexible/evolving schemas, and very high write throughput, often at the cost of relaxing strict ACID guarantees in favor of **eventual consistency (BASE)**:

- **Basically Available**
- **Soft state**
- **Eventually consistent**

9.2 Categories of NoSQL Databases

Category	Data Model	Examples	Typical Use Case
Key-Value Store	Simple key → value pairs	Redis, DynamoDB, Riak	Caching, session storage
Document Store	Semi-structured documents (JSON/BSON)	MongoDB, CouchDB	Content management, catalogs
Column-Family Store	Rows with dynamic, sparse columns grouped into families	Cassandra, HBase, Bigtable	Time-series, large-scale analytics
Graph Database	Nodes and edges with properties	Neo4j, Amazon Neptune	Social networks, recommendation engines, fraud detection

9.3 NewSQL

NewSQL systems aim to provide the scalability of NoSQL while retaining full ACID guarantees and SQL query support, e.g., **Google Spanner**, **CockroachDB**, **VoltDB** — often using techniques like automatic sharding, distributed consensus (e.g., Paxos/Raft), and globally synchronized clocks.

9.4 Data Warehousing and OLAP

- **OLTP (Online Transaction Processing)**: Handles day-to-day operational transactions — many short read/write operations, normalized schemas.
- **OLAP (Online Analytical Processing)**: Handles complex analytical queries over historical data — read-heavy, often denormalized for speed.
- **Data Warehouse**: A subject-oriented, integrated, time-variant, non-volatile collection of data used for analysis and decision-making, typically loaded via **ETL (Extract, Transform, Load)** processes from operational systems.

- **Star Schema**: A central **fact table** (containing measures/metrics) surrounded by **dimension tables** (containing descriptive attributes), denormalized for fast querying.
- **Snowflake Schema**: A star schema where dimension tables are further normalized into sub-dimensions.
- **OLAP Operations**: *Roll-up* (aggregate to a coarser level), *Drill-down* (go to finer detail), *Slice* (fix one dimension), *Dice* (select a sub-cube), *Pivot* (rotate axes for a different view).

10 Database Security

10.1 Security Threats

- **Loss of integrity**: Unauthorized modification of data.
- **Loss of availability**: Data or the system becomes inaccessible (e.g., denial-of-service).
- **Loss of confidentiality**: Unauthorized disclosure of sensitive data.

10.2 Authentication and Authorization

- **Authentication**: Verifying the identity of a user (passwords, multi-factor authentication, certificates).
- **Authorization**: Determining what an authenticated user is permitted to do.

10.2.1 Discretionary Access Control (DAC)

Access rights (privileges) are granted to users on specific objects, typically via `GRANT`/`REVOKE` statements. Privileges can be granted `WITH GRANT OPTION`, allowing recipients to further grant them to others.

```
GRANT SELECT, UPDATE ON Employee TO 'hr_user'@'%' WITH GRANT OPTION;
REVOKE UPDATE ON Employee FROM 'hr_user'@'%';
```

10.2.2 Mandatory Access Control (MAC)

Access is controlled by fixed security classifications (e.g., Top Secret, Secret, Confidential, Unclassified) assigned to both subjects (users) and objects (data). Common models: **Bell-LaPadula** (no read-up, no write-down — preserves confidentiality) and **Biba** (integrity-focused, opposite rules).

10.2.3 Role-Based Access Control (RBAC)

Privileges are assigned to **roles**, and roles are assigned to users, greatly simplifying administration compared to assigning privileges to each user individually.

```
CREATE ROLE analyst_role;
GRANT SELECT ON SalesData TO analyst_role;
GRANT analyst_role TO 'jane'@'%';
```

10.3 SQL Injection

SQL injection is one of the most common and dangerous vulnerabilities, occurring when untrusted user input is concatenated directly into a SQL query string, allowing an attacker to alter query logic.

Vulnerable example (do not use):

```
query = "SELECT * FROM Users WHERE username = '" + input + "'"
```

An attacker entering `' OR '1'='1` could bypass authentication logic entirely.

Mitigations: - Use **parameterized queries / prepared statements** (never string-concatenate user input into SQL). - Apply the **principle of least privilege** to database accounts used by applications. - Validate and sanitize all user input. - Use stored procedures carefully (they help but are not a silver bullet if they build dynamic SQL internally).

10.4 Encryption

- **Encryption at rest**: Protects stored data files/backups from being read if physically accessed.
- **Encryption in transit** (e.g., TLS/SSL): Protects data moving between client and server from eavesdropping.
- **Transparent Data Encryption (TDE)**: Database-level encryption that is invisible to applications.
- **Column-level / field-level encryption**: Encrypts specific sensitive columns (e.g., credit card numbers).

10.5 Auditing

Database auditing tracks and logs access to and modification of data, supporting compliance requirements (e.g., HIPAA, GDPR, PCI-DSS) and forensic investigation after a breach.

10.6 Backup and Recovery Strategy

- **Full backup**: A complete copy of the database.
- **Incremental backup**: Only data changed since the last backup.
- **Differential backup**: All data changed since the last *full* backup.
- **Point-in-time recovery**: Restoring a database to its exact state at a specific past moment, typically by combining a full backup with transaction logs.
- **Hot/Online backup**: Taken while the database remains operational.
- **Cold/Offline backup**: Taken while the database is shut down.

11 Popular Relational Database Systems — A Comparative Overview

11.1 MySQL

An open-source RDBMS (now owned by Oracle), widely used for web applications. Default storage engine is **InnoDB**, which supports transactions, foreign keys, and row-level locking via MVCC. Known for ease of use, strong community support, and tight integration with the LAMP/LEMP stack.

11.2 PostgreSQL

An open-source, object-relational database known for standards compliance, extensibility (custom types, functions, extensions like PostGIS for spatial data), and advanced features such as full MVCC, table inheritance, and sophisticated indexing (GiST, GIN, BRIN).

11.3 Oracle Database

A commercial enterprise-grade RDBMS known for advanced features: Real Application Clusters (RAC) for high availability, Automatic Storage Management (ASM), extensive partitioning, and strong support for large-scale OLTP and OLAP workloads.

11.4 Microsoft SQL Server

A commercial RDBMS tightly integrated with the Microsoft ecosystem (.NET, Azure), offering T-SQL as its procedural extension to SQL, Integration Services (SSIS) for ETL, and Analysis Services (SSAS) for OLAP.

11.5 SQLite

A lightweight, serverless, self-contained, file-based relational database engine embedded directly into applications — no separate server process. Extremely popular for mobile apps, embedded systems, and local application storage.

11.6 Comparison Summary

Feature	MySQL	PostgreSQL	Oracle	SQL Server	SQLite
License	Open-source (GPL)	Open-source	Commercial	Commercial	Public domain
Best for	Web apps	Complex queries, extensibility	Enterprise OLTP/OLAP	Windows/.NET ecosystems	Embedded/local apps
Concurrency model	MVCC (InnoDB)	Full MVCC	MVCC	Lock-based + optional MVCC	File-level locking

Feature	MySQL	PostgreSQL	Oracle	SQL Server	SQLite
Replication	Native	Native (streaming/logical)	Data Guard	Always On	N/A (single file)

12 Advanced and Emerging Topics

12.1 Object-Relational and Object-Oriented Databases

Object-Oriented DBMSs (OODBMS) store data as objects, closely mirroring object-oriented programming, and support inheritance, encapsulation, and complex/nested object structures natively (avoiding the "object-relational impedance mismatch"). **Object-Relational DBMSs (ORDBMS)**, such as PostgreSQL, extend the relational model with object-oriented features like user-defined types, inheritance, and array/composite columns while retaining SQL and relational guarantees.

12.2 Temporal Databases

Temporal databases manage time-varying data explicitly:

- **Valid time**: The time period during which a fact is true in the real world.
- **Transaction time**: The time period during which a fact is recorded in the database.
- **Bitemporal databases**: Track both valid time and transaction time simultaneously, enabling full historical auditing and "as-of" queries.

12.3 Spatial and Geographic Databases

Store and query geometric/geographic data (points, lines, polygons) using specialized indexing (R-trees, Quad-trees) and operators for spatial relationships (contains, intersects, within a distance). Examples: PostGIS (PostgreSQL extension), Oracle Spatial.

12.4 In-Memory Databases

Store the primary copy of data in RAM rather than on disk, drastically reducing latency for reads/writes. Durability is typically achieved via snapshotting and/or write-ahead logging to disk in the background. Examples: Redis, SAP HANA, VoltDB, MemSQL/SingleStore.

12.5 Graph Databases in Depth

Graph databases model data as **nodes** (entities), **edges** (relationships, which can be directed and have properties), and properties on both. They excel at queries involving deep or variable-length relationship traversal (e.g., "friends of friends," shortest path, fraud rings) that would require many expensive joins in a relational model. Query languages include **Cypher** (Neo4j) and **Gremlin** (Apache TinkerPop).

12.6 Data Mining and Big Data Integration

- **Data Mining**: Discovering patterns (associations, clusters, classifications, anomalies) in large datasets, often applied on top of data warehouses.

- **Big Data (the 3/5 Vs)**: Volume (scale of data), Velocity (speed of generation/processing), Variety (structured, semi-structured, unstructured), and often extended with Veracity (data quality/trustworthiness) and Value.
- **Hadoop Ecosystem**: HDFS (distributed file storage), MapReduce/Spark (distributed processing), Hive (SQL-like querying over Hadoop data).
- **Data Lakes**: Store raw data in its native format (structured, semi-structured, unstructured) at scale, in contrast to the curated, schema-on-write approach of a traditional data warehouse (data lakes are typically schema-on-read).

12.7 Database Trends

- **Cloud-native/Database-as-a-Service (DBaaS)**: Managed database offerings (Amazon RDS/Aurora, Google Cloud SQL/Spanner, Azure SQL Database) that abstract away provisioning, patching, and scaling.
- **Serverless databases**: Automatically scale (including to zero) based on demand, billing per usage rather than for fixed provisioned capacity.
- **Vector databases**: Store and index high-dimensional vector embeddings for similarity search, increasingly used to support AI/ML applications such as semantic search and retrieval-augmented generation (e.g., Pinecone, Milvus, pgvector).

13 Solved Problems and Worked Examples

This chapter reinforces the concepts covered in earlier chapters through fully worked problems, the kind commonly encountered in coursework and interviews.

13.1 Problem Set 1: ER Modeling

Problem: A university wants to track Students, Courses, and Instructors. Each student can enroll in many courses, and each course can have many students (with a grade recorded per enrollment). Each course is taught by exactly one instructor, but an instructor may teach multiple courses. Draw the ER model conceptually and convert it to relations.

Solution:

Entities: `Student(StudentID, Name, Email)`, `Course(CourseID, Title, Credits)`, `Instructor(InstructorID, Name, Office)`.

Relationships: `Teaches` (Instructor 1:N Course), `Enrolls` (Student M:N Course, with attribute `Grade`).

Converting to relations:

```
CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(100),
    Office VARCHAR(20)
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    Title VARCHAR(100),
    Credits INT,
    InstructorID INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100)
);

CREATE TABLE Enrolls (
    StudentID INT,
    CourseID INT,
    Grade CHAR(2),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
```

```
);
```

Note the M:N `Enrolls` relationship became its own table (with a composite primary key), while the 1:N `Teaches` relationship was implemented by placing `InstructorID` as a foreign key on the "many" side (`Course`).

13.2 Problem Set 2: Normalization

Problem: Consider the relation `OrderDetails(OrderID, ProductID, ProductName, ProductPrice, Quantity, CustomerID, CustomerName, CustomerCity)` with the following functional dependencies:

- `OrderID → CustomerID, CustomerName, CustomerCity`
- `ProductID → ProductName, ProductPrice`
- `CustomerID → CustomerName, CustomerCity`
- `{OrderID, ProductID} → Quantity`

Normalize this relation to 3NF.

Solution:

Step 1 — Check 1NF: All attributes are atomic; the relation is in 1NF.

Step 2 — Check 2NF: The primary key is `{OrderID, ProductID}`. `ProductName` and `ProductPrice` depend only on `ProductID` (part of the key) — this is a partial dependency, violating 2NF. Likewise `CustomerID`, `CustomerName`, `CustomerCity` depend only on `OrderID`.

Decompose to remove partial dependencies:

- `Order(OrderID, CustomerID)`
- `Product(ProductID, ProductName, ProductPrice)`
- `OrderLine(OrderID, ProductID, Quantity)`
- `Customer(CustomerID, CustomerName, CustomerCity)` — split out further below.

Step 3 — Check 3NF: In the intermediate `Order` table, if we had kept `CustomerName` and `CustomerCity`, they would be transitively dependent on `OrderID` through `CustomerID` (`OrderID → CustomerID → CustomerName`). This is why `Customer` was split into its own table above — removing the transitive dependency.

Final 3NF schema:

```sql
CREATE TABLE Customer (CustomerID INT PRIMARY KEY, CustomerName VARCHAR(100), CustomerC
CREATE TABLE Product (ProductID INT PRIMARY KEY, ProductName VARCHAR(100), ProductPrice
CREATE TABLE Orders (OrderID INT PRIMARY KEY, CustomerID INT REFERENCES Customer(Custom
CREATE TABLE OrderLine (
    OrderID INT, ProductID INT, Quantity INT,
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
```

13.3 Problem Set 3: Relational Algebra

Given `Employee(EmpID, Name, DeptID, Salary)` and `Department(DeptID, DeptName)`:

Problem A: Find the names of employees earning more than 5000.

Solution: `π(Name)(σ(Salary > 5000)(Employee))`

Problem B: Find the names of employees along with their department names.

Solution: $\pi_{Name,\ DeptName}(Employee \bowtie Department)$ — a natural join on `DeptID`, followed by projection.

Problem C: Find department names that have no employees.

Solution: $\pi_{DeptName}(Department) - \pi_{DeptName}(Employee \bowtie Department)$ — using set difference between all department names and those that appear with at least one employee.

13.4 Problem Set 4: Transaction Schedules and Serializability

Problem: Determine whether the following schedule is conflict-serializable:

```
T1: Read(A)  Write(A)              Read(B)  Write(B)
T2:          Read(A)  Write(A)
```

Interleaved order: `R1(A), W1(A), R2(A), W2(A), R1(B), W1(B)`

Solution: Build a precedence graph. `W1(A)` happens before `R2(A)` (conflict: both access A, one writes) → edge T1 → T2. There is no reverse conflict (T2 has no operation after T1 resumes that conflicts with T1's later operations, since T2 doesn't touch B). The precedence graph has a single edge T1 → T2 with no cycle, so **the schedule is conflict-serializable**, equivalent to the serial schedule T1 followed by T2.

13.5 Problem Set 5: Deadlock Detection

Problem: Transaction T1 holds a lock on resource X and requests a lock on resource Y. Transaction T2 holds a lock on resource Y and requests a lock on resource Z. Transaction T3 holds a lock on resource Z and requests a lock on resource X. Is there a deadlock?

Solution: Build a wait-for graph: T1 → T2 (waiting for Y), T2 → T3 (waiting for Z), T3 → T1 (waiting for X). This forms a cycle **T1 → T2 → T3 → T1**, so **yes, this is a deadlock**. The DBMS's deadlock detector would choose a victim transaction (e.g., the one with the least work done, or the youngest transaction) to abort and roll back, releasing its locks so the others can proceed.

13.6 Problem Set 6: Two-Phase Commit Walkthrough

Problem: A distributed transaction updates data at Site A and Site B, coordinated by Site C. Site A votes "yes," but Site B fails to respond within the timeout. What happens?

Solution: In Phase 1 (voting), the coordinator (Site C) waits for votes from all participants. Since Site B does not respond (timeout is treated as an implicit “no” / failure), the coordinator cannot get unanimous “yes” votes. In Phase 2, the coordinator sends an **ABORT** message to all participants, including Site A (even though A voted “yes”). Site A must undo any tentative changes. This illustrates why 2PC is a blocking, all-or-nothing protocol — a single non-responsive participant forces global abort (or, if the coordinator itself fails after collecting votes, participants may block indefinitely waiting for the decision).

13.7 Problem Set 7: Index Selection

Problem: A table `Orders(OrderID, CustomerID, OrderDate, Status)` has 10 million rows. Queries frequently filter by `CustomerID` and `OrderDate` together, and separately by `Status` (which has only 4 distinct values: Pending, Shipped, Delivered, Cancelled). What indexing strategy would you use?

Solution: - A **composite B+-tree index** on `(CustomerID, OrderDate)` supports the combined filter efficiently, following the leftmost-prefix rule (it also supports queries filtering on `CustomerID` alone, but not on `OrderDate` alone). - For `Status`, since it has very low cardinality (4 distinct values), a traditional B+-tree index would have poor selectivity (each value matches ~25% of rows) and may not be used by the optimizer. A **bitmap index** on `Status` is far more appropriate here, especially if `Status` is often combined with other conditions using AND/OR. - The primary key `OrderID` already has a clustered/unique index by default in most engines.

Appendix A: Glossary of Key Terms

ACID — Atomicity, Consistency, Isolation, Durability; the guarantees of a reliable transaction.

Attribute — A named property of an entity or column of a relation.

BASE — Basically Available, Soft state, Eventually consistent; the consistency model common in many NoSQL systems.

Candidate Key — A minimal set of attributes that uniquely identifies a tuple.

CAP Theorem — States a distributed system can guarantee at most two of Consistency, Availability, and Partition tolerance simultaneously.

Cardinality — The number of tuples in a relation, or the maximum number of relationship instances an entity can participate in (in ER modeling).

Deadlock — A cycle of transactions each waiting for a resource held by another.

Foreign Key — An attribute referencing the primary key of another (or the same) relation.

Functional Dependency — A constraint that one attribute set uniquely determines another.

Index — An auxiliary data structure that speeds up data retrieval.

Isolation Level — The degree to which a transaction is isolated from the effects of concurrent transactions.

Join — A relational operation combining tuples from two relations based on a related attribute.

MVCC — Multiversion Concurrency Control; maintains multiple data versions to allow non-blocking reads.

Normalization — The process of organizing data to minimize redundancy and avoid update anomalies.

Primary Key — The chosen candidate key that uniquely identifies each tuple in a relation; cannot be NULL.

Referential Integrity — The constraint that foreign key values must correspond to existing primary key values (or be NULL).

Schema — The overall structure/design of a database, as opposed to its actual data (instance).

Serializability — A schedule is serializable if its effect is equivalent to some serial execution of the same transactions.

Transaction — A logical unit of work that must execute completely or not at all.

View — A virtual table defined by a stored query over one or more base tables.

WAL — Write-Ahead Logging; the principle that log records must be written before corresponding data changes are flushed to disk.

Appendix B: Quick Reference — Common SQL Commands

Command	Purpose
`CREATE TABLE`	Define a new table
`ALTER TABLE`	Modify an existing table's structure
`DROP TABLE`	Remove a table and its data
`SELECT`	Retrieve data
`INSERT INTO`	Add new rows
`UPDATE`	Modify existing rows
`DELETE FROM`	Remove rows
`GRANT` / `REVOKE`	Manage privileges
`COMMIT` / `ROLLBACK`	End a transaction
`CREATE INDEX`	Speed up retrieval on a column
`CREATE VIEW`	Define a virtual table
`JOIN`	Combine rows from multiple tables
`GROUP BY` / `HAVING`	Aggregate and filter groups
`UNION` / `INTERSECT` / `EXCEPT`	Combine result sets

End of Book